ARM PrimeCell Driver

CLCD v2.3 (PL110)

API Specification

PL110 -ISPC-1000-D

Open Access

Proprietary notice

Document confidentiality status

This document is Open Access. This document has no restriction on distribution.

Product status

The information in this document is Final (information on a developed product).

Web address

http://www.arm.com

Feedback

ARM limited welcomes feedback on both the product, and the documentation.

Feedback on this document

If you have any comments on about this document, please send email to errata@arm.com giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

Document Summary

Document Issue	D
PrimeCell driver identifier	CLCD
Code Version	2.3
PrimeCell identifier(s)	PL110

Contents

1.1 Change control

1.1.1 Change history

Issue	Date	Change
A		Beta release of **PrimeCell** CLCD
B	11/05/2001	Revised document
C	29/11/2001	ISR made public
D	17/12/2001	Changed type of apCLCD_PowerUp() and apCLCD_WaitVComp() functions. Macro apCLCD_NO_POWER_UP_DELAY added.

2 SUMMARY

The PrimeCell Color Liquid Crystal Display Controller (CLCDC) peripheral is an Advanced Microcontroller Bus Architecture (AMBA) slave block that connects to the Advanced High-performance Bus (AHB). It is an AMBA compliant System-on-Chip peripheral developed, tested and licensed by ARM.

The PrimeCell CLCD Driver supports all the major features of the Controller, and allows the driver to be configured to operate a wide range of specific LCD panels.

3 FEATURES

The main features supported by the PrimeCell CLCD driver are:

- Single and dual panel Super Twisted Nematic (STN) monochrome displays with 4 or 8 bit interfaces.
- Single and dual panel STN colour displays.
- Thin Film Transistor (TFT) colour displays.
- Resolution up to 1024x768.
- 15 level grey-scale, 3375 colour STN and 32K colour TFT display modes.
- 1, 2 or 4 bits-per-pixel (bpp) monochrome palettes for STN displays.
- 1, 2, 4 or 8bpp colour palettes for STN and TFT displays.
- 16bpp direct true-colour for STN and TFT displays.
- 24bpp direct true-colour for TFT displays.

4 DRIVER USAGE

4.1 Build options

4.1.1 apCLCD_USER_SETUP

Defining the pre-processor constant apCLCD_USER_SETUP in apclcd.h enables the compilation of calls to three optional user-written functions. These are called at the start of initialisation, at power-up and at power-down. This allows configuration and hardware specific features to be configured. See section 0 below.

4.1.2 apCLCD_NO_POWERUP_DELAY

Defining the pre-processor constant apCLCD_NO_POWER_UP_DELAY disables pause in apCLCD_PowerUp() function before the panel powering up. This constant should be defined in appltatfm.h for correct power up of the Sharp panels.

4.2 Driver Initialisation

- Call apCLCD_Initialize() to initialise the PrimeCell specific data. Specify the peripheral base address and interrupts. No initialisation data is passed, so the final parameter is unused.
- Call apCLCD_PanelInitialize() to initialise the PrimeCell with LCD specific data. The settings for the LCD are specified in an apCLCD_sDisplayParams structure. A pointer to this is passed to this function. The structure must be created and initialised by the caller. The elements of the apCLCD_sDisplayParams structure and their usage are described below in section 0.
- Call apCLCD_PaletteSet() to load the palette data if the display is being used at less than 16 bits per pixel. Above 16bpp the data is written directly to the CLCD.
- Call apCLCD_PowerUp() to switch on the display. DMA refresh of the display from the display memory area then starts, and data can be displayed on the CLCD by writing to the display memory.

4.3 Drawing Function Support

The LCD PrimeCell updates the display, using DMA transparent to the User, from a specified area of memory.

Functions are provided that allow the drawing functions to map onto the DMA memory so that the data destined for the display can be written in an appropriate format.

The functions are:

- apCLCD_BppGet()

- apCLCD_DMAFrameBufferGet()
- apCLCD_LineWidthGet()
- apCLCD_LinesPerScreenGet()
- apCLCD_PixelOrderGet()

apCLCD_DMAFrameBufferSet() sets a DMA memory address to be used by the PrimeCell. This allows the use of multiple buffering. The screen image can be written to an area of memory in advance and then passed into the driver to be displayed using this function. The new memory area is then used for DMA refresh of the LCD. An interrupt can be generated to use this facility efficiently.

Care should be taken selecting the memory to use for DMA, as there may be speed and access limitations.

4.4 Interrupts and Events

The use of interrupts is not necessary for the CLCD PrimeCell and driver, however should the user need them, they can be enabled and disabled using apCLCD_InterruptEventsDisable(), apCLCD_InterruptEventsEnable() and cleared using apCLCD_InterruptsDisable().

apCLCD_RegisterCallback() is used to register an interrupt handler which is then called by the driver whenever an interrupt has occurred and been acknowledged.

Interrupts for four events can be enabled:

- FIFO underflow – One of the two DMA FIFOs has been read when empty.
- AHB Master error – The AMBA bus AHB master has encountered a bus error from a slave.
- DMA base address update – The current base address registers have been updated. A new address can now be loaded.
- Vertical compare – There are four vertical regions defined for the LCD, and an interrupt can be enabled to occur at the start of any one of these. The events are selected using the apCLCD_VcompEventsEnable() function, and any one of the following can be specified:
  - Vertical Synchronisation – The start of the horizontal synchronisation lines.
  - Back Porch – The start of the region of inactive lines at the start of the frame.
  - Active Video – The start of the visible displayed screen lines.
  - Front Porch – The start of the region of inactive lines at the end of the frame.

The function apCLCD_WaitVComp() is provided which waits for the currently selected Vertical Compare event. This function polls the PrimeCell status without using interrupts.

4.5 Other Features

A number of support functions are provided:

- apCLCD_BppSet() – Changes the Bits Per Pixel operation of the CLCD after initialisation.
- apCLCD_SetRefresh() – Changes the refresh rate of the CLCD to be after initialisation.

- apCLCD_PowerUp() – Powers up the LCD safely.
- apCLCD_PowerDown() – Powers down the LCD safely.

5 DETAILS

- Peak performance obtained in tests: N/A
- ROM size (RO code + RO data) (release build): 3044 bytes
- RAM size (ZI data) per instance (release build): 56 bytes
- Does this driver support multiple PrimeCells?: YES
- Can these be detected at run-time? NO
- Does this driver support both endian-nesses?[1] YES
- Does this driver support Thumb?[2] YES

6 RELEASED FILES

`PL110 –ISPC-1000-D`	This API document
`PL110 –TREP-1000`	Test Results document
`apclcd/apclcd.h`	Public header file
`apclcd/clcd.h`	Private header file
`apclcd/clcd.c`	Source code
`apclcd/clcdtst.c`	Self-test module
`apclcd/apdraw.h`	Self-test module
`apclcd/draw.h`	Drawing functions header file
`apclcd/apdraw.c`	Drawing functions module used by clcdtst.c
`apclcd/clr8x8.c`	Test font for drawing functions
`apclcd/clr8x8.h`	Test font declaration

1 To support both endian-nesses, the driver must be designed to be re-compiled in little-endian or big-endian form.

2 The driver may include ARM code, but this must interwork with a Thumb build.

7 PUBLIC API

7.1 Type Definitions

7.1.1 apCLCD_eBpp

Color LCD Bits Per Pixel Enumerator.

Declaration

```
typedef enum apCLCD_eBpp
```

Elements

apCLCD_1BPP	1 bpp
apCLCD_2BPP	2 bpp
apCLCD_4BPP	4 bpp
apCLCD_8BPP	8 bpp
apCLCD_16BPP	16 bpp
apCLCD_24BPP	24 bpp (TFT panel only)

Remarks

Definitions of the possible LCD bits per pixel for the LCD. Used in the display parameters structure. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize(). Also used as a parameter to the function apCLCD_BppSet().

7.1.2 apCLCD_eClockSource

Color LCD clock source Enumerator.

Declaration

```
typedef enum apCLCD_eClockSource
```

Elements

apCLCD_EXTERNAL_CLOCK	Clock is external source
apCLCD_INTERNAL_CLOCK	Clock is HCLCK

Remarks

The input clock for the color LCD can be taken either from HCLK or from an external source. Used in the display parameters structure. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize().

7.1.3 apCLCD_eColorOrder

Color LCD RGB format Enumerator.

Declaration

```
typedef enum apCLCD_eColorOrder
```

Elements

apCLCD_BGR	Pixels are in B/G/R format
apCLCD_RGB	Pixels are in R/G/B format

Remarks

Definitions of the possible pixel component orderings for the color LCD used in the display parameters structure. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize().

7.1.4 apCLCD_eDataDrive

Color LCD Data Drive Edge Enumerator.

Declaration

```
typedef enum apCLCD_eDataDrive
```

Elements

apCLCD_CLCP_F	Data driven on falling edge of CLCP
apCLCD_CLCP_R	Data driven on rising edge of CLCP

Remarks

Definitions of the possible LCD data drive edges for the color LCD. Used in the display parameters structure. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize().

7.1.5 apCLCD_eError

General errors for this module.

Declaration

```
typedef enum apCLCD_eError
```

Elements

apCLCD_INCORRECT_WIDTH	The LCD width must be a multiple of 16 pixels
apCLCD_INVALID_BPP	Bits Per Pixel cannot be achieved, i.e. greater than 24

	for STN, or greater than 16 for TFT displays
apCLCD_INVALID_DISPLAY_TYPE	Display type options incompatible or wrong
apCLCD_NO_AC_BIAS	STN displays must have AC bias
apCLCD_NO_SETTING	Required configuration value missing
apCLCD_PALETTE_TOO_BIG	The specified number of entries is more than 256
apCLCD_VCOMP_TIMEOUT	Vcomp bit was not set during the time-out period

7.1.6 apCLCD_eHSync_Active

Color LCD HSync Active Enumerator.

Declaration

```
typedef enum apCLCD_eHSync_Active
```

Elements

apCLCD_LCDLP_H	LcdLP pin is active high, inactive low
apCLCD_LCDLP_L	LcdLP pin is active low, inactive high

Remarks

Definitions of the possible LCD HSync active states for the color LCD used in the display parameters structure. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize().

7.1.7 apCLCD_eInterrupts

Color LCD Interrupt Type Definitions.

Declaration

```
typedef enum apCLCD_eInterrupts
```

Elements

apCLCD_NO_INTERRUPTS	Interrupt mask with all interrupts cleared
apCLCD_FUF	FIFO underflow
apCLCD_MBERR	AHB Master error
apCLCD_NEXT_BASE_UP	DMA base address update
apCLCD_VCOMP	Vertical compare
apCLCD_ALL_INTERRUPTS	Interrupt mask with all interrupts set

Remarks

Definitions of the possible types of LCD event that can be set in the PrimeCell status register. These events can raise an interrupt if it is so enabled. Alternatively the status register can be polled to detect the event. Pass a bitmask formed by ORing together these options to the apCLCD_InterruptEventsEnable() and

apCLCD_InterruptEventsDisable() functions as required.

These are also used by the interrupt handler as the codes identifying the interrupt type, and are passed as the parameter to the callback function.

7.1.8 apCLCD_eOEnable

Color LCD Data Enable Output Enumerator.

Declaration

```
typedef enum apCLCD_eOEnable
```

Elements

apCLCD_CLAC_H	Data Enable Output (CLAC) pin is active high in TFT mode
apCLCD_CLAC_L	Data Enable Output (CLAC) pin is active low in TFT mode

Remarks

Definitions of the possible LCD output enable settings for the color LCD used in the display parameters structure. For STN displays so this setting has no effect and the CLAC output pin is used for the AC Bias signal. For TFT displays it is used as the data output enable, and this setting is used. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize().

7.1.9 apCLCD_ePixelOrder

Color LCD pixel order format Enumerator.

Declaration

```
typedef enum apCLCD_ePixelOrder
```

Elements

apCLCD_BIG_ENDIAN_BYTES	Leftmost pixel is at bit 7
apCLCD_LITTLE_ENDIAN_BYTES	Leftmost pixel is at bit 0

Remarks

Definition of the pixel order within a byte for the color LCD. Used in the display parameters structure. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize(). The ordering of pixels within a word is set to match the compilation options. Rebuild the driver with a bigendian target set to change the LCD word order selected.

7.1.10 apCLCD_eReload

Color LCD reload time Enumerator.

Declaration

```
typedef enum apCLCD_eReload
```

Elements

apCLCD_RELOAD_EARLY	Load more data when FIFO has 4 slots
apCLCD_RELOAD_LATE	Load more data when FIFO has 8 slots

Remarks

Definitions of the possible DMA reload timings for the color LCD. Used in the display parameters structure. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize().

7.1.11 apCLCD_eSTN_CType

Color LCD STN Color Enumerator.

Declaration

```
typedef enum apCLCD_eSTN_CType
```

Elements

apCLCD_STN_COLOR	STN LCD is color
apCLCD_STN_MONO	STN LCD is monochrome

Remarks

Definition of color and mononchrome STN LCD. Used in the display parameters structure. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize(). Ignored if the display type selected (apCLCD_eType) is TFT.

7.1.12 apCLCD_eSTN_Iface

Color LCD STN x-Bits Per Pixel Interface Enumerator.

Declaration

```
typedef enum apCLCD_eSTN_Iface
```

Elements

apCLCD_STN_4BIT	4 bit parallel interface
apCLCD_STN_8BIT	8 bit parallel interface

Remarks

Definitions of the possible parallel interfaces for a monochrome STN LCD used in the display parameters structure. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize(). Ignored if the display type selected is TFT.

7.1.13 apCLCD_eSTN_Panel

Color LCD STN Dual Panel Enumerator.

Declaration

```
typedef enum apCLCD_eSTN_Panel
```

Elements

apCLCD_STN_DUAL	Selects a dual panel STN LCD
apCLCD_STN_SINGLE	Selects a single panel STN LCD

Remarks

Definition of the number of panels on the color LCD. Used in the display parameters structure. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize(). Ignored if the display type selected is TFT.

7.1.14 apCLCD_eType

Color LCD Panel Type Enumerator. Definitions of the possible LCD panel types for the color LCD. Used in the display parameters structure.

Declaration

```
typedef enum apCLCD_eType
```

Elements

apCLCD_STN	Selects an Super Twisted Nematic display
apCLCD_TFT	Selects a Thin Film Transistor display

Remarks

Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize().

7.1.15 apCLCD_eVComp_ITime

Color LCD VComp Interrupt Time Enumerator.

Declaration

```
typedef enum apCLCD_eVComp_ITime
```

Elements

apCLCD_ACTIVE_V	Start of Active Video
apCLCD_BACK_P	Start of Back Porch
apCLCD_FRONT_P	Start of Front Porch
apCLCD_VSYNC	Start of VSync

Remarks

Definitions of the possible timing for the LCD vertical compare (VComp) interrupt. Used in the display parameters structure. Care is needed if selecting start of Front or Back Porch. With STN displays these values are typically set to zero and in this case the corresponding Vcomp events and their interrupts can never happen, even if enabled.

Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize(). Used as an argument to the apCLCD_EnableVCompEvents() function.

7.1.16 apCLCD_eVSync_Active

Color LCD VSync Active Enumerator.

Declaration

```
typedef enum apCLCD_eVSync_Active
```

Elements

apCLCD_LCDFP_H	LcdFP pin is active high, inactive low
apCLCD_LCDFP_L	LcdFP pin is active low, inactive high

Remarks

Definitions of the possible LCD VSync active states for the color LCD. Used in the display parameters structure. Set the appropriate value in the apCLCD_sDisplayParams structure passed to apCLCD_PanelInitialize().

7.1.17 apCLCD_rCallback

Type definition for the User callback handler used by the CLCD interrupt handler.

Declaration

```
typedef void (*apCLCD_rCallback) (apCLCD_eInterrupts Condition)
```

Remarks

Establishes the User provided function that is called whenever a CLCD interrupt happens. When the callback function is called, the interrupt event has already been acknowledged.

As an example of an interrupt:

A double-buffer display swap could be performed when an LCD next address base update (LNBU) interrupt is enabled and generated.

If the reason code is apCLCD_NEXT_BASE_UP it can be used to switch the CLCD to use a different display buffer.

The current DMA base address can be read using the function apCLCD_DMAFrameBufferGet(). This is now free memory area that can be reused, and can be stored locally. A second (already prepared) memory area can then be selected for use on the next refresh cycle using the function apCLCD_DMAFrameBufferSet().

The driver’s interrupt handler clears the interrupt. You do not need to clear it in your callback code.

7.1.18 apCLCD_rUserFunction

Type definition for User functions for performing LCD panel specific operations.

Declaration

```
typedef void (*apCLCD_rUserFunction) (void)
```

Remarks

Because of variations between panels, there may be setup that must be done before anything else, before power up or after power up, which cannot be generalized. If these operations are needed, provide functions for them and set them in the apCLCD_sDisplayParams structure passed to the function apCLCD_PanelInitialize() as rPreamble, rSwitchOff and rSwitchOn. They are only called if the pre-processor constant apCLCD_USER_SETUP is defined.

7.1.19 apCLCD_sDisplayParams

Color LCD Display Parameters Structure.

Declaration

```
struct apCLCD_sDisplayParams
{
    apCLCD_eType         eType;
    apCLCD_eColorOrder   eColorOrder;
    apCLCD_eReload       eReload;
    apCLCD_eClockSource  eClockSource;
    UWORD32              ACBias;
    apCLCD_eVSync_Active eVSyncActive;
    apCLCD_eHSync_Active eHSyncActive;
    apCLCD_eDataDrive    eDataDrive;
    apCLCD_eOEnable      eTftClac;
    UWORD32              ClleDelay;

    apCLCD_eSTN_CType    eColorType;
    apCLCD_eSTN_Iface    eInterface;
    apCLCD_eSTN_Panel    ePanel;
    UWORD32              Clock;
    UWORD32              VSync;
    UWORD32              VFront;
    UWORD32              VBack;
    UWORD32              HSync;
    UWORD32              HFront;
    UWORD32              HBack;

    UWORD32              Refresh;
    apCLCD_eVComp_ITime  eVCInterTime;

    UWORD32              DMABase;
    UWORD32              LineWidth;
    UWORD32              NumLines;
    apCLCD_ePixelOrder   ePixelOrder;
    apCLCD_eBpp          eBpp;

#ifndef apCLCD_NO_POWERUP_DELAY
```

```
    UWORD32             PowerUpDel;
#endif
    UWORD32             PowerDownDel;

#if defined apCLCD_USER_SETUP
    apCLCD_rUserFunction rPreamble;
    apCLCD_rUserFunction rSwitchOff;
    apCLCD_rUserFunction rSwitchOn;
#endif
}
```

Elements

The parameters in this structure are grouped according to usage.

- Parameters used only at initialisation. They are not saved internally in the driver.

eType	LCD display type (STN or TFT)
eColorOrder	The palette can be stored either RGB or BGR as needed by the LCD
eReload	FIFO reload timing. This can be when 4 or 8 locations become available
eClockSource	LCD clock source. This can either be HCLCK or an external clock source
ACBias	Number of line clock periods between each toggle of AC-bias pin. Should be 0 for STN displays
eVSyncActive	LcdFP pin active state
eHSyncActive	LcdLP pin active state
eTftClac	Data enable output (CLAC) pin setting. TFT mode only
ClleDelay	Delay before line end pulse (0 to disable)

- Parameters that are needed if the refresh rate is to be changed after initialisation by calling the function apCLCD_RefreshSet(). They are not held in the driver, and if this feature is needed, either the user must preserve the parameter structure passed to the driver via the apCLCD_PanelInitialize() function, or the driver should be modified to save the values internally.

eColorType	STN LCD color type, Color or Monochrome
eInterface	STN LCD x-bit interface, 4-bit or 8-bit
ePanel	STN LCD single or double panel
Clock	LCD controller clock in Hz
Vsync	Vertical sync pulse width, use a value of 1 for STN displays
Vfront	Vertical front porch, use zero for STN displays
Vback	Vertical back porch, use zero for STN displays
Hsync	Horizontal sync pulse width
Hfront	Horizontal front porch
Hback	Horizontal back porch
Refresh	Refresh rate in frames per second. This is changed after initialisation if the function apCLCD_RefreshSet() is called to change the configuration. This function also sets this to the actual refresh rate set up

- Parameters needed by drawing functions to address DMA memory. They are held internally in the driver.

DMABase	Base address of screen memory used by the PrimeCell DMA
LineWidth	Line length. Must be a multiple of 16 pixels
NumLines	Total active screen height
ePixelOrder	Pixel ordering within a byte
eBpp	Bits Per Pixel. This is changed after initialisation if the configuration is altered by calling the apCLCD_BppSet() function

- Panel specific time delays for powering the LCD. They are stored in the driver.

PowerUpDel	Delay between LcdEn & LcdPwr on
PowerDownDel	Delay between LcdPwr off & LcdEn off

- Panel specific functions supplied by the User to handle start-up and close-down. The preamble is used only at initialisation. Is not stored by the driver, but the switch on and off functions are.

rPreamble	Panel specific function to be called before all else
rSwitchOff	Panel specific power down
rSwitchOn	Panel specific power up

Remarks

This is the definition of the structure holding all the static and variable color LCD panel display parameters. The User should create an instance of this structure, and initialize it to reflect the LCD device in use and the configuration in which it is to be used. A pointer to this structure is then passed to the driver in apCLCD_PanelInitialize(). The driver keeps a pointer to this data in it's internal state structure so it should not be destroyed if the driver needs to access it again. The circumstances under which this might happen are described below.

Most of the data is only used at initialisation, so there are no run-time speed implications.

User drawing functions need access to the parameters that describe how the LCD PrimeCell addresses the DMA memory area. To facilitate this, these parameters are copied into the driver's internal state structure so that they can be accessed after initialisation without problem. Other values however are needed only if the refresh rate is changed after the panel has been initialised. These values are not copied, so if the User wants to change the refresh rate after initialisation, the structure must be preserved.

The Bits Per Pixels (apCLCD_eBpp) setting of the LCD can be changed after initialisation, and all the data needed is held within the driver and the parameter structure is not needed.

The current system, where the driver keeps a pointer to this structure in its own state structure, may not be suitable for some environments and memory configurations. In this case the pointer could then be removed from the state structure, provided provision is made for storing additional parameters, in particular for the case where it is required to alter refresh rate after initialisation.

7.1.20 apCLCD_sPalette

Color LCD Palette Structures.

Declaration

```
typedef union apCLCD_xPalette {
    apCLCD_sRGB_Palette sRgb;
    UBYTE8              Grey;
    UWORD32             Entry;
} apCLCD_sPalette
```

Remarks

Each palette entry for the color LCD is either a greyscale value or an RGB color. A greyscale entry is a byte quantity, but only the top bits are defined for a physical palette entry. An RGB color is a structure. The definition gives the combined greyscale and RGB palette structure.

The definition gives the combined greyscale and RGB palette structure.

7.1.21 apCLCD_sRGB_Palette

Color LCD RGB Palette Structure.

Declaration

```
struct apCLCD_sRGB_Palette {

    UBYTE8   Red;
    UBYTE8   Green;
    UBYTE8   Blue;
}
```

Elements

Blue	Blue component in top five bits of byte
Green	Green component in top five bits of byte
Red	Red component in top five bits of byte

Remarks

Each RGB palette entry for the color LCD consists of a byte value for each of the three colors, red, green, and blue. The top five bits of each color are written directly to the palette. The average of the low three bits is used to decide whether to set the Intensity bit (which is used only for TFT displays). STN displays use only the top 4 bits of these 5-bit fields.

The PrimeCell can handle displays that use either RGB/BGR color ordering. The PrimeCell does this, and the palette structure remains unchanged.

7.2 External Functions

7.2.1 apCLCD_BppGet

Retrieves the current setting of bits per pixel for the LCD.

Declaration

```
PUBLIC apCLCD_eBpp apCLCD_BppGet(apOS_CLCD_oId oId)
```

Parameters

oId	Selects the LCD data to be referenced

Return Value

The enumerated type value describing the bits per pixel.

Remarks

This value is held within the Driver's internal data structure, and is typically used by drawing functions for mapping the LCD onto the DMA memory region.

7.2.2 apCLCD_BppSet

Set up the Bits Per Pixel to be used.

Declaration

```
PUBLIC apError apCLCD_BppSet(apOS_CLCD_oId oId, UWORD32 Bpp)
```

Parameters

oId	Selects the LCD data to be referenced
Bpp	The Bits Per Pixel required

Return Value

apCLCD_INVALID_BPP	An unobtainable Bits Per Pixel was requested
apERR_NONE	The parameters are compatible

Remarks

Stores the bits per pixel in the Driver's internal state structure so that it can be used by any drawing functions that might need it later. Modifies the Control register to apply the setting. For dual panel displays, the Lower Panel base address is re-evaluated.

7.2.3 apCLCD_DMAFrameBufferGet

Retrieves the base address of the current DMA memory block.

Declaration

```
PUBLIC UWORD32 apCLCD_DMAFrameBufferGet(apOS_CLCD_oId oId)
```

Parameters

oId	Selects the LCD data to be referenced

Return Value

Address of the current DMA memory.

Remarks

The DMA memory address is held in the LCD Driver's internal data structure. It is typically required by drawing functions that write to this area. The implementation assumes that for dual-panel LCDs the Upper and Lower memory areas are contiguous.

NOTE: The driver must be modified if it is to use non-contiguous memory.

7.2.4 apCLCD_DMAFrameBufferSet

Establishes the DMA Frame buffer address(es).

Declaration

```
PUBLIC void apCLCD_DMAFrameBufferSet(apOS_CLCD_oId oId, UWORD32 DMABase)
```

Parameters

oId	Selects the LCD data to be referenced
DMABase	The base address of the LCD DMA panel memory. If the device dual panel, this is the address of the upper panel

Remarks

Sets up the DMA Frame buffer address(es). If the LCD in use is dual panel STN device, the second frame buffer address is calculated and set up. This function is typically used to implement multiple buffering in conjunction with LNBUINTR interrupts. As implemented, the Driver assumes that for dual panel LCDs the pair of buffers occupies contiguous memory.

7.2.5 apCLCD_StateSizeGet

Size of the Driver's internal data structure.

Declaration

```
PUBLIC WORD32 apCLCD_StateSizeGet(void)
```

Return Value

Size of the Driver's internal data structure.

Remarks

The Driver maintains a structure for each LCD initialized, which contains data that may be needed after initialization. Typically this is not accessible to the user. It may be that the driver must operate without making use of any private data of global scope, i.e. Using heap space. In this case the user must allocate this data and pass it to the Driver using the opaque data type apOS_CLCD_oId in the apCLCD_Initialize() function. To do this the size of the data structure is needed by the calling program.

This function is conditionally compiled, and only exists if the value apOS_NO_STATIC_STATE is defined.

7.2.6 apCLCD_Initialize

Initialize the CLCD Driver.

Declaration

```
PUBLIC void apCLCD_Initialize(apOS_CLCD_oId                   oId,
                              apOS_System_eBaseAddress        eBase,
                              UWORD32                         Interrupts,
                              CONST apOS_INT_oInterruptSource *pSources,
                              void                            *pInitial)
```

Parameters

oId	Selects the LCD data to be referenced
eBase	Base address of CLCD registers
Interrupts	Number of interrupts used for the LCD being initialized (1)
pSources	Source of interrupt apOS_INT_LMCLCD
pInitial	Pointer to initialisation data structure (unused)

Remarks

This Function must be called before any other action is carried out on the CLCD PrimeCell.

7.2.7 apCLCD_InterruptEventsDisable

Disables interrupt events within the LCD controller.

Declaration

```
PUBLIC void apCLCD_InterruptEventsDisable(apOS_CLCD_oId oId, UWORD32 Events)
```

Parameters

oId	Selects the LCD data to be referenced
Events	The interrupt events to be disabled. Formed by ORing together the required apCLCD_eInterrupts values

Remarks

The interrupt events specified are disabled. The equivalent bits in the LCD Interrupt Enable PrimeCell register are cleared.

This function must only be called after both apCLCD_Initialize() and apCLCD_PanelInitialize() have been called.

7.2.8 apCLCD_InterruptEventsEnable

Enables interrupt events within the LCD controller.

Declaration

```
PUBLIC void apCLCD_InterruptEventsEnable(apOS_CLCD_oId oId, UWORD32 Events)
```

Parameters

old	Selects the LCD data to be referenced
Events	The interrupt events to be enabled. Formed by ORing together the required apCLCD_eInterrupts values

Remarks

The interrupt events specified are enabled. The equivalent bits in the LCD Interrupt Enable PrimeCell register are set.

This function must only be called after both apCLCD_Initialize() and apCLCD_PanelInitialize() have been called.

7.2.9 apCLCD_InterruptsDisable

Disable all CLCD PrimeCell interrupts.

Declaration

```
PUBLIC void apCLCD_InterruptsDisable(apOS_CLCD_oId oId)
```

Parameters

old	Selects the LCD data to be reference

Remarks

Clears all interrupt bits in the LCD Interrupt enable register, and clears all pending interrupts.

7.2.10 apCLCD_LineWidthGet

Retrieve the horizontal size of the LCD active area.

Declaration

```
PUBLIC UWORD32 apCLCD_LineWidthGet(apOS_CLCD_oId oId)
```

Parameters

old	Selects the LCD data to be referenced

Return Value

LCD active area width.

Remarks

This value is held within the Driver's internal data structure, and is typically used by drawing functions for mapping the LCD onto the DMA memory region.

7.2.11 apCLCD_LinesPerScreenGet

Retrieve the vertical size of the LCD active area.

Declaration

```
PUBLIC UWORD32 apCLCD_LinesPerScreenGet(apOS_CLCD_oId oId)
```

Parameters

oId	Selects the LCD data to be referenced

Return Value

LCD active area height.

Remarks

This value is held within the Driver's internal data structure, and is typically used by drawing functions for mapping the LCD onto the DMA memory region.

7.2.12 apCLCD_PaletteSet

Defines the palette for the specified number of contiguous entries (starting at zero). The palette is used for either greyscale or RGB color output.

Declaration

```
PUBLIC apError apCLCD_PaletteSet(apOS_CLCD_oId         oId,
                                 CONST apCLCD_sPalette sPalette[],
                                 UWORD32               Entries)
```

Parameters

oId	Selects the LCD data to be referenced
sPalette	An array of physical level palette entry structures of either greyscale or RGB components
Entries	The number of physical palette levels to define using the entries in the palette supplied

Return Value

apCLCD_PALETTE_TOO_BIG	More than 256 entries specified
apERR_NONE	Parameters OK

Remarks

For each of the entries defined in the physical color level palette, palette[x] is converted and written to the appropriate 16-bit field in the CLCD Palette, where 'x' is the current palette entry to be written, $0 \leq x < \text{Entries}$.

For greyscale output only the first 16 entries of the red component of the palette is used. The green and blue components and the remaining 240 palette entries are undefined.

For 1bpp color output only the first 2 entries of the RGB components of the palette are used. The remaining 254 palette entries are unused.

For 2bpp color output only the first 4 entries of the RGB components of the palette are used. The remaining 252 palette entries are unused.

For 4bpp color output only the first 16 entries of the RGB components of the palette are used. The remaining 240 palette entries are unused.

For 8bpp color output the RGB components of all 256 palette entries are used.

For 16 or 24bpp, no palette is used, and the values are written directly to the PrimeCell.

In all cases, the palette entries and components beyond those that are used for the LCD controller mode

can still be set even though they are unused.

7.2.13 apCLCD_PanelInitialize

Initializes the LCD panel using values from the apCLCD_sDisplayParams structure passed in. The contents of this structure are defined above, and the usage of the parameters in the PrimeCEll is described in ARM PrimeCell Color LCD Controller (PL110) Technical Reference Manual.

Declaration

```
PUBLIC apError apCLCD_PanelInitialize(apOS_CLCD_oId         oId,
                                      apCLCD_sDisplayParams *pParams)
```

Parameters

oId	Selects the LCD data to be referenced
pParams	Pointer to the initialisation parameter structure

Return Value

apERR_NONE	The parameters are complete and compatible
apCLCD_INCORRECT_WIDTH	Display width is not a multiple of 16 pixels
apCLCD_NO_SETTING	A required initialisation value is missing or invalid
apCLCD_INVALID_DISPLAY_TYPE	Either an unknown display type is specified, Or an STN display is selected with options that conflict
apCLCD_NO_AC_BIAS	STN displays specified with an AC bias of zero. If the bias is zero the display will degrade with time

Remarks

This Function must be called after apCLCD_Initialize(), but before any other operations are carried out on the CLCD PrimeCell.

7.2.14 apCLCD_PixelOrderGet

Retrieves the pixel ordering in use for the LCD.

Declaration

```
PUBLIC apCLCD_ePixelOrder apCLCD_PixelOrderGet(apOS_CLCD_oId oId)
```

Parameters

oId	Selects the LCD data to be referenced

Return Value

The pixel ordering in use.

Remarks

This value is held within the Driver's internal data structure, and is typically used by drawing functions for mapping the LCD onto the DMA memory region.

7.2.15 apCLCD_PowerDown

Powers-down and disables the color LCD controller.

Declaration

```
PUBLIC void apCLCD_PowerDown(apOS_CLCD_oId oId)
```

Parameters

old	Selects the LCD data to be referenced

Remarks

The LCD controller is powered-down by clearing bits in the Control Register. First LcdPwr is cleared and after a suitable delay, typically 20ms, LcdEn is cleared to switch off the LCD controller. The delay may vary between LCD panels, and its value is held internally by the driver. The PrimeCell manual recommends 20ms, but individual display data sheets may specify other values.

7.2.16 apCLCD_PowerUp

Enables and powers-up the color LCD controller.

Declaration

```
PUBLIC apError apCLCD_PowerUp(apOS_CLCD_oId oId)
```

Parameters

old	Selects the LCD data to be referenced

Return Value

apERR_NONE	Power-up sequence completed without errors
apCLCD_VCOMP_TIMEOUT	Vcomp bit was not set during time-out period

Remarks

The LCD controller is enabled by setting bits in the LCD Control Register. LcdEn is set first, and then after a delay, typically of 20ms, the LCD controller is powered-up by setting the LcdPwr bit. The clocks are started when LcdEn is set. LCD displays require that this sequence is followed or the panel may be degraded. The delay may vary between LCD panels, so its value is held in the parameter structure. The PrimeCell manual recommends 20ms, but display data sheets may specify other values. This delay can be omitted by defining the constant apCLCD_NO_POWER_UP_DELAY in appItatfm.h.

7.2.17 apCLCD_RegisterCallback

Registers the user callback which is called every time a CLCD interrupt occurs. To disable this and prevent any further calls to the function register the null routine pointer aRNULL.

Declaration

```
PUBLIC void apCLCD_RegisterCallback(apOS_CLCD_oId oId, apCLCD_rCallback rCallback)
```

Parameters

old	Selects the LCD data to be referenced
rCallback	The function to be called on interrupts

7.2.18 apCLCD_RefreshSet

Sets the PrimeCell Driver clock divisor to give a refresh rate close to that requested.

Declaration

```
PUBLIC UWORD32 apCLCD_RefreshSet(apOS_CLCD_oId oId, UWORD32 Refresh)
```

Parameters

oId	Selects the LCD data to be referenced.
Refresh	The refresh rate requested, in Frames per Second

Return Value

The refresh rate actually established.

Remarks

Uses the values in the Parameter structure to generate a clock divisor. This will produce an LCD Panel Clock frequency that gives a refresh rate close to that requested. Since the clock divisor may well be a small integer, there can be a large difference between the rate requested and that actually set. This is particularly so for high refresh rates. Because of this the resulting refresh rate is computed and saved in the parameter structure and returned to the caller.

7.2.19 apCLCD_VCompEventsEnable

Declaration

```
PUBLIC BOOL apCLCD_VCompEventsEnable(apOS_CLCD_oId       oId,
                                     apCLCD_eVComp_ITime eVCompEvent)
```

Parameters

oId	Selects the LCD data to be referenced
eVCompEvent	The requested reason for Vertical Compare Interrupts

Return Value

If the settings requested preclude VComp interrupts happening FALSE is returned. Otherwise TRUE.

Remarks

Sets the Vertical Compare interrupt can be generated at one of four timing positions:

- Start of Vertical Sync.
- Start of Back Porch.
- Start of Active Video.
- Start of Front Porch.

The behaviour of the PrimeCell is such that there are two configurations for which no interrupt occurs.

- The interrupt is selected for the start of front porch and the front porch value is zero.
- The interrupt is selected for the start of back porch and the back porch value is zero.

The requested setting is always selected even if it precludes Vcomp interrupts.

7.2.20 apCLCD_WaitVComp

Wait for the requested number of occurrences of a Vertical Compare event.

Declaration

```
PUBLIC apError apCLCD_WaitVComp(apOS_CLCD_oId oId, UWORD32 VCompCount)
```

Parameters

oId	selects the LCD data to be referenced

Return Value

apERR_NONE	Vcomp bit was set requested number of times, or cannot be set
apCLCD_VCOMP_TIMEOUT	Vcomp bit was not set during time-out period

Remarks

This function works by polling the status register for VComp events. Because of this the User must ensure that the settings for VComp are as needed, using apCLCD_VCompEventsEnable(), before calling this function.

The function returns immediately if the settings are such that no Vcomp events can occur. If Vcomp bit has not been set during 10000 polling periods, the function returns error code.

7.2.21 apCLCD_IntHandler

Standard interrupt handler for MMCI module.

Declaration

```
PUBLIC void apCLCD_IntHandler(CONST apOS_INT_oInterruptSource oInterruptId,
                              UWORD32                         DeviceId)
```

Parameters

oInterruptId	The ID of the interrupt
DeviceId	Identifier for the instance of the driver

Return Value

None

Remarks

Only an interrupt dispatcher should call this function.

7.2.22 apCLCD_RawISR

This is the raw interrupt handler for the module, to be called directly from the interrupt vector

Declaration

```
PUBLIC IRQ void apCLCD_RawISR(void)
```

Parameters

None

Return Value

None

Remarks

This routine should only be executed as a branch from the IRQ vector.

7.3 Macros

7.3.1 apCLCD_USER_SETUP

Declaration

```
#define apCLCD_USER_SETUP
```

Remarks

Because of variations between specific LCD displays there may be some initialisation which cannot be generalized, and that must be done before or during power up. Provision is made for three user-defined functions:

- A preamble, called before any initialisation of the PrimeCell.
- A switch off function, called before power-up and at the end of the standard power-down sequence.
- A switch on function, called after the standard power-up is complete.

An example is that the clock source for the LCD can be selected by an external clock multiplexor, which is selected using the CLKSEL bit in the PrimeCell control register. In this case, it may be necessary to initialise this external device before the CLCD PrimeCell is initialised. In this case a user function would be written to carry out the initialisation, and called via the preamble.

If these functions are not needed, the definition of apCLCD_USER_SETUP can be removed. The code that calls them will not then be compiled.

7.3.2 apCLCD_NO_POWER_UP_DELAY

Declaration

```
#define apCLCD_NO_POWER_UP_DELAY
```

Remarks

Defining this macro disables pause in apCLCD_PowerUp() function before the panel powering up. This constant should be defined in appltatfm.h to omit pause delay for correct power up of the Sharp panels.

APPENDIX A – HOW TO DETERMINE CLCD SETTINGS

A.1 Determining Settings for an LCD

Most of the settings needed for a specific LCD are readily determined. Some are likely to be described in the LCD manufacturer's data sheet, such as:

- Display type (TFT or STN, single or dual panel).
- Display size.
- Colour or Monochrome.
- 4 or 8 bit parallel interface for monochrome STN displays. (If the Display is TFT or Colour STN, this should be set to 4 bit).
- For the signals whose sense can be selected in the PrimeCell, the sense to use.
- The delays required on power-up and power-down (nominally 20ms).
- AC Bias for STN displays, which should be non-zero. Unused if TFT.

The clock frequency, clock source, and DMA memory base address depend on how the PrimeCell has been incorporated into the User's design. User-defined configuration code called at start-up, power-up and power-down also depends on the design. This code allows any required control actions to be taken such as defining clocks or setting up other devices that work in conjunction with the CLCD PrimeCell.

The Bits Per Pixel depends on the application in which the display is used.

The refresh rate can be determined experimentally by finding a setting at which the display is flicker-free. If such a setting cannot be found, the clock rate must be increased. It may be possible to adjust the rate slightly using the settings for horizontal or vertical porches and Synchronisation pulse width. These vary with display types.

For STN displays:

- The Vertical Synchronization Pulse Width should be small, typically 1, and Vertical Front and Back Porches should be 0. Increasing these values can lead to poor contrast.
- Horizontal Synchronization Width, Front and Back Porches. The best values can be established by experiment. It might be possible to adjust the refresh rate slightly by altering them. The latency on the data path to ensure that data can propagate down the FIFO path in the LCD interface imposes minimum values. These are: HSW = 3, HBP = 5, HFP = 5.

For TFT displays:

- Altering Horizontal and Vertical Synchronization Pulse Widths and porch settings alters the position of the active area on the display.